

relax_orm

Audit technique — bugs et points d'attention

Couverture : moteur de synchronisation, couche de persistance, schéma, annotations

Version analysée : 0.1.7 — 28 juin 2026

Table des matières

1. Synthèse

Ce document recense les bugs et points d'attention identifiés lors de la revue du code source du package `relax_orm` (moteur de synchronisation, couche de persistance Drift, définition de schéma, et annotations). L'analyse porte sur le code réel des fichiers fournis, pas sur la documentation publique du package.

Onze constats sont recensés, classés par sévérité. Aucun n'est bloquant pour un usage local sans synchronisation réseau ; deux (ORM-1 et SY-1) méritent une correction avant tout usage impliquant respectivement un import de données volumineux avec UI réactive, ou une synchronisation réseau de données utilisateur critiques.

ID	Titre	Fichier	Sévérité
SY-1	Correspondance push ↔ confirmation serveur non vérifiée	<code>sync_engine.dart</code>	Critique
SY-2	Aucune confirmation possible pour les suppressions partielles	<code>sync_engine.dart</code> / <code>sync_adapter.dart</code>	Élevée
SY-3	Résolution de conflit asymétrique entre push et pull, non documentée	<code>sync_engine.dart</code>	Moyenne
SY-4	<code>`clear()`</code> ne permet pas de cibler une seule table	<code>offline_queue.dart</code>	Basse
ORM-1	<code>`rawBatchInsert`</code> ne notifie pas les streams réactifs	<code>relax_database.dart</code>	Critique
ORM-2	Identifiant généré automatiquement non récupérable par l'appelant	<code>collection.dart</code> / <code>table_schema.dart</code>	Élevée
ORM-3	<code>`upsert()`</code> : lecture puis écriture sans transaction (fenêtre de course)	<code>collection.dart</code>	Moyenne
ORM-4	Couplage implicite entre le format de la queue de sync et le schéma de colonnes	<code>collection.dart</code>	Moyenne
ORM-5	Résolution de schéma par recherche linéaire malgré une structure indexée	<code>relax_db.dart</code>	Basse
ORM-6	Dépendance à un contrat interne non garanti de Drift	<code>relax_database.dart</code>	Basse
GEN-1	Écart entre l'API annotée documentée et l'implémentation réelle (génération de schéma)	<code>table_schema.dart</code>	Élevée

2. Détail des constats

SY-1 — Correspondance push ↔ confirmation serveur non vérifiée

Sévérité: **Critique**

Fichier : sync_engine.dart

Description

Dans ``_SyncRegistration.pushOperations``, la liste ``confirmed`` retournée par ``SyncAdapter.push(entities)`` est réécrite localement sans aucune vérification de longueur ni d'identité avec ``entities`` :

```
final confirmed = await config.adapter.push(entities); if (confirmed.isNotEmpty)
{   for (final entity in confirmed) {       await _writeLocal(db, tableName,
entity);   } }
```

Impact

- Si l'implémentation de ``SyncAdapter.push`` côté développeur renvoie une liste partielle (certaines entités rejetées côté serveur sans lever d'exception), ces entités sont quand même marquées comme synchronisées par ``_pushChanges`` juste après, via ``completeAll(pending.map(...))``.
- Le contrat ne garantit ni l'ordre, ni la cardinalité, ni un identifiant explicite associant chaque entité confirmée à l'opération en attente d'origine.
- Conséquence concrète : une opération peut être retirée de la queue alors qu'elle n'a jamais réellement atteint le serveur — perte silencieuse de données de synchronisation.

Recommandation

- Faire retourner par ``SyncAdapter.push`` une structure associant explicitement chaque entité poussée à son statut (succès/échec), par exemple une ``Map<Object, SyncPushResult<T>>`` indexée par clé primaire, plutôt qu'une ``List<T>`` positionnelle.
- Dans ``_pushChanges``, ne marquer ``complete`` que les opérations dont la confirmation a été explicitement reçue ; les autres doivent rester ``pending`` ou passer à ``failed``.

SY-2 — Aucune confirmation possible pour les suppressions partielles

Sévérité: **Élevée**

Fichier : sync_engine.dart / sync_adapter.dart

Description

Contrairement à ``push``, ``pushDeletes(ids)`` ne retourne rien :

```
Future<void> pushDeletes(List<Object> ids);
```

Impact

- Si une partie seulement des suppressions réussit côté serveur (ex. 3 ids sur 5), il n'existe aucun moyen pour l'adaptateur de communiquer cet état partiel à l'engine sans lever une exception globale qui annulerait aussi les 3 suppressions réussies.
- Toutes les opérations de suppression du lot sont marquées ``complete`` ensemble dès que ``pushDeletes`` retourne sans exception.

Recommandation

- Faire retourner à `pushDeletes` la liste des ids effectivement confirmés par le serveur, sur le même modèle que la correction proposée pour SY-1.

SY-3 — Résolution de conflit asymétrique entre push et pull, non documentée

Sévérité: **Moyenne**

Fichier : sync_engine.dart

Description

Le `ConflictResolver` n'est appliqué que dans `applyPull` (lecture distante). Lors de l'écriture du retour serveur après un `push` réussi, `\_writeLocal` est appelé avec `resolver: null`, donc la version serveur écrase toujours la version locale sans passer par le resolver configuré.

```
// applyPull await _writeLocal(db, tableName, remoteEntity, resolver:
config.conflictResolver); // pushOperations (after push confirmation) await
_writeLocal(db, tableName, entity); // resolver implicite = null
```

Impact

- Comportement probablement intentionnel (un push confirmé doit refléter l'état serveur), mais cette asymétrie n'est documentée nulle part dans le code ni dans la doc publique.
- Un développeur tiers pourrait supposer à tort que `conflictResolver` s'applique uniformément à toute écriture locale issue du réseau.

Recommandation

- Documenter explicitement cette asymétrie dans la docstring de `SyncConfig.conflictResolver` et dans le README.

SY-4 — `clear()` ne permet pas de cibler une seule table

Sévérité: **Basse**

Fichier : offline_queue.dart

Description

La méthode supprime l'intégralité de la table interne `\_relax\_sync\_queue`, toutes tables confondues :

```
Future<void> clear() async { await _db.rawDelete(_table, where: '1 = 1', whereArgs:
[]); }
```

Impact

- Impossible de purger la queue d'une seule collection sans perdre les opérations en attente des autres collections enregistrées.

Recommandation

- Ajouter un paramètre optionnel `String? tableName` à `clear()`, sur le même modèle que `resetFailed`.

ORM-1 — `rawBatchInsert` ne notifie pas les streams réactifs

Sévérité: **Critique**

Fichier : relax_database.dart

Description

Les opérations unitaires (`rawInsert`, `rawUpdate`, `rawDelete`) déclarent explicitement la table affectée via `updates: {tableRef(table)}`, ce qui permet à Drift de notifier les `watch()` actifs. `rawBatchInsert` utilise `batch()` avec des `customStatement` bruts, sans déclarer aucune table affectée :

```
Future<void> rawBatchInsert(String table, List<Map<String, Object?>> rows) async
{
  if (rows.isEmpty) return;
  await batch((b) {
    for (final row in rows)
    {
      final sql = 'INSERT INTO $table (...) VALUES (...)';
      b.customStatement(sql, [...row.values]);
    }
  });
}
```

Impact

- Tout appel à `Collection.addAll()` — donc à `rawBatchInsert` — n'émettra probablement aucune notification vers les listeners actifs de `watchAll()` / `watch()` sur la même table.
- Cas d'usage directement concerné : import initial d'un catalogue volumineux (ex. import de centaines/milliers d'enregistrements en un seul `addAll()`) suivi d'un affichage réactif via `watchAll()` — l'UI ne se rafraîchira pas tant qu'une autre écriture (même unitaire) ne survient pas sur la table.
- C'est, parmi tous les constats de cet audit, celui qui a le plus de chances d'être rencontré tôt par un développeur utilisant le package en conditions réelles.

Recommandation

- Passer `updates: {tableRef(table)}` à l'appel `batch()`, ou déclarer la table affectée explicitement après le batch (ex. en émettant une notification manuelle sur le `StreamQuery` concerné).
- Ajouter un test d'intégration qui appelle `addAll()` puis vérifie qu'un `watchAll()` actif émet bien une nouvelle valeur.

ORM-2 — Identifiant généré automatiquement non récupérable par l'appelant

Sévérité: Élevée

Fichier : collection.dart / table_schema.dart

Description

Lorsque l'id d'une entité est `null` à l'insertion, `TableSchema.entityToRow` génère un UUID, mais uniquement dans la version SQL du `row` :

```
if (value == null && col.isPrimaryKey && col.type == ColumnType.text) {
  value = const Uuid().v4();
}
```

Impact

- `Collection.add(entity)` retourne `void`. L'objet `entity` original (généralement immuable en Dart, champs `final`) passé par l'appelant conserve son id `null` après l'appel.
- L'appelant n'a aucun moyen direct de récupérer l'id généré sans requêter à nouveau la base (ex. via une recherche par un autre champ unique, si disponible).

Recommandation

- Faire retourner par `add()` l'entité reconstruite (`T` au lieu de `void`), comme c'est déjà fait en interne pour la valeur passée à `_queueSync` (`updatedEntity`).

ORM-3 — `upsert()` : lecture puis écriture sans transaction (fenêtre de course)

Sévérité: **Moyenne**

Fichier : collection.dart

Description

La méthode enchaîne un `get()` puis un `add()` ou `update()` sans transaction englobante :

```
Future<void> upsert(T entity) async {    final exists = await
get(_schema.getPrimaryKeyValue(entity));    if (exists != null) {        await
update(entity);    } else {        await add(entity);    } }
```

Impact

- Si une écriture concurrente sur la même clé primaire survient entre le `get()` et le `add()` — notamment un pull du `SyncEngine` tournant en parallèle sur la même table — `add()` peut lever une violation de contrainte `PRIMARY KEY` au lieu de basculer vers une mise à jour.
- Scénario concret et non exotique : un favori modifié localement pendant qu'une synchronisation périodique (`autoSyncInterval`) écrit la même entité reçue du serveur.

Recommandation

- Envelopper la lecture et l'écriture dans une transaction Drift (`db.transaction`), ou utiliser une requête SQL native `INSERT ... ON CONFLICT DO UPDATE` pour rendre l'opération atomique.

ORM-4 — Couplage implicite entre le format de la queue de sync et le schéma de colonnes

Sévérité: **Moyenne**

Fichier : collection.dart

Description

La donnée stockée dans la queue offline est le *row SQL-encodé* (`entityToRow`), pas l'entité Dart, par choix explicite documenté en commentaire :

```
// Store the SQL-encoded row (DateTime → int, bool → int, ...) so the queue's // JSON
serialization never fails on non-primitive Dart values. final data =
_schema.entityToRow(entity);
```

Impact

- Le format stocké dans `\_relax\_sync\_queue` dépend directement de `ColumnDef.toSql()` au moment de l'écriture.
- Si le type d'une colonne change entre deux versions de l'app (ex. `text` → `blob`), les opérations déjà en attente dans la queue au moment de la migration seront décodées avec le nouveau schéma — rien ne versionne le format des données stockées, le décodage peut échouer silencieusement ou produire une valeur incorrecte.
- Risque concret pour tout projet amené à faire évoluer son schéma de données sur plusieurs mois/années avec des utilisateurs ayant des opérations en attente au moment de la mise à jour.

Recommandation

- Ajouter un numéro de version de schéma à chaque `SyncOperation` enqueued, et invalider/migrer explicitement les opérations dont la version ne correspond plus au schéma courant au démarrage.

ORM-5 — Résolution de schéma par recherche linéaire malgré une structure indexée**Sévérité:** Basse**Fichier :** relax_db.dart**Description**

Les schémas sont stockés dans une `Map<Type, TableSchema>` indexée par `schema.runtimeType` (c'est-à-dire `TableSchema<User>`), mais la résolution `\_findSchema<T>()` parcourt la map en boucle avec un test `is TableSchema<T>` plutôt que d'utiliser une clé directe :

```
TableSchema<T> _findSchema<T>() { for (final schema in _schemas.values) { if
(schema is TableSchema<T>) return schema; } throw StateError(...); }
```

Impact

- Complexité $O(n)$ au lieu de $O(1)$ à chaque appel de `db.collection<T>()`. Sans impact mesurable pour un nombre de tables faible (cas de la quasi-totalité des apps mobiles), mais révèle une incohérence entre la structure de données choisie et l'algorithme réellement utilisé.

Recommandation

- Indexer la map par un type réifiable correspondant à `T` directement (ex. via une clé `String` basée sur le nom de type), ou conserver la recherche linéaire en documentant que c'est un choix volontaire pour la simplicité.

ORM-6 — Dépendance à un contrat interne non garanti de Drift**Sévérité:** Basse**Fichier :** relax_database.dart**Description**

`\_RawTableReference`, utilisé pour le suivi des changements de table par Drift, lève des `UnsupportedError` sur deux accesseurs hérités :

```
@override DatabaseConnectionUser get attachedDatabase => throw
UnsupportedError('Not needed for stream tracking'); @override Table get asDslTable =>
throw UnsupportedError('Not needed for stream tracking');
```

Impact

- Fonctionne aujourd'hui parce que le chemin de code de Drift utilisé pour le tracking de streams n'appelle pas ces accesseurs. Aucune garantie de compatibilité ascendante : une future version de Drift qui les invoquerait dans un nouveau chemin (ex. logging de debug interne) ferait échouer `rawWatch` / `rawWatchOne` partout dans l'application, potentiellement de façon difficile à diagnostiquer.

Recommandation

- Épingler une plage de version stricte pour `drift` dans `pubspec.yaml`, et ajouter un test de fumée exécuté en CI à chaque mise à jour de la dépendance Drift pour détecter une régression sur ce point précis.

GEN-1 — Écart entre l'API annotée documentée et l'implémentation réelle (génération de schéma)

Sévérité: **Élevée**

Fichier : table_schema.dart

Description

Le commentaire du code source indique explicitement une transition de phase non finalisée :

```
/// In Phase 1a, schemas are defined manually. In Phase 2, the code generator /// will
produce these automatically from annotated classes.
```

Impact

- Les annotations `@RelaxTable()`, `@PrimaryKey()`, `@Column()`, `@Ignore()` existent comme classes utilisables, mais aucun fichier fourni dans cet audit ne montre le générateur (`relax\_orm\_generator`) qui les lit pour produire un `TableSchema` automatiquement.
- Pour un usage en l'état, tout `TableSchema` doit être écrit manuellement, ce qui contredit l'exemple `@RelaxTable()` mis en avant dans la documentation publique (README pub.dev) comme méthode principale.
- Période de transition entre API manuelle et API générée : c'est typiquement le moment où les changements d'API non rétrocompatibles sont les plus probables.

Recommandation

- Clarifier dans la documentation publique l'état exact d'avancement (Phase 1a vs Phase 2) pour éviter qu'un nouvel utilisateur ne s'appuie sur un workflow annoté qui n'est pas encore opérationnel.

3. Points forts confirmés à la lecture du code

Ces éléments ne sont pas des bugs mais méritent d'être notés : ils correspondent à des choix d'implémentation déjà solides, observés directement dans le code plutôt que dans la documentation marketing du package.

- Le coalescing de la queue offline est implémenté à deux niveaux cohérents (à l'écriture dans `OfflineQueue.enqueue`, et au push dans `SyncOperation.coalesce`), avec des règles de fusion identiques aux deux endroits — un point que beaucoup d'implémentations similaires laissent diverger avec le temps.
- Le watermark serveur (`serverTime`) est capturé avant l'appel réseau (`startedAt`) en fallback, ce qui évite de manquer des changements survenus côté serveur pendant le round-trip de synchronisation.
- Les écritures locales du pull sont englobées dans une transaction Drift unique : un échec en cours de pull ne laisse pas la base dans un état partiellement synchronisé.

- La logique de dead-letter sur les opérations en échec (`retryCount`` incrémenté, seuil `maxRetries`` respecté avant abandon) est correctement implémentée et empêche une boucle de réessai infinie.
- L'interface `SyncAdapter<T>` est totalement découplée du modèle ORM et de ses annotations — la logique réseau reste entièrement à la charge du développeur, ce qui évite un couplage rigide entre format de persistance locale et protocole distant.
- Le chiffrement transparent s'accompagne d'un outil de vérification qui inspecte réellement les octets du fichier sur disque (`debugCheckEncryption``) plutôt que de se contenter de confirmer la configuration — y compris la détection explicite d'un cas mal configuré (clé fournie mais fichier resté en clair).
- Le logger interne (`RelaxLogger``) a un coût nul lorsqu'il est désactivé (vérification précoce avant toute construction d'objet), et segmente automatiquement les messages longs pour éviter la troncature dans les consoles habituelles (~1000 caractères).
- La séparation des conversions de types (`ColumnDef.toSql`` / `fromSql``) est centralisée à un seul endroit du code plutôt que dispersée dans chaque appelant.

4. Ordre de traitement recommandé

- **Priorité 1** — Corriger ORM-1 (notification manquante sur ``addAll``) avant tout usage en production impliquant un import initial de données volumineuses suivi d'un affichage réactif.
- **Priorité 2** — Corriger SY-1 et SY-2 (pertes silencieuses de confirmation push/delete) avant d'activer la synchronisation réseau sur des données utilisateur critiques (ex. favoris).
- **Priorité 3** — Documenter explicitement GEN-1 (état réel de la génération de schéma par annotations) pour éviter qu'un nouvel utilisateur ne base son intégration sur un workflow non encore opérationnel.
- **Priorité 4** — Traiter ORM-2 et ORM-3 (retour d'id généré, atomicité de l'upsert) avant d'exposer le package à des scénarios concurrents (sync en arrière-plan + écritures utilisateur simultanées).
- **Priorité 5** — Les points restants (SY-3, SY-4, ORM-4, ORM-5, ORM-6) sont à traiter sur la feuille de route normale ; aucun n'est bloquant pour un usage initial maîtrisé.

5. Périmètre de l'audit

Fichiers analysés : `conflict_resolver.dart`, `offline_queue.dart`, `sync_adapter.dart`, `sync_engine.dart`, `sync_operation.dart`, `sync_status.dart`, `column_def.dart`, `table_schema.dart`, `relax_logger.dart`, `relax_database.dart`, `collection.dart`, `query_builder.dart`, `relax_db.dart`, `relax_orm_json.dart`, `annotations.dart`, `relax_orm.dart`, `relax_orm_annotations.dart`.

Cette revue est statique (lecture de code) et ne remplace pas une suite de tests d'intégration. Plusieurs constats (notamment ORM-1) gagneraient à être confirmés par un test automatisé reproduisant le scénario décrit.